Tu responsabilidad y mapa del código

Archivo: src/validation.py | Función o clase: validate_atm_record_detailed | Inicio: línea 281

Tu parte define qué registros son válidos, explica sus errores y los guarda con seguridad. Maki limpia y calcula parámetros; Avilix integra y coordina. Los módulos se adaptaron en equipo; responsabilidad técnica no equivale a autoría exclusiva de cada línea.

Mapa: src/validation.py contiene ATMTransaction, DigitalTransaction y validadores detallados. src/persistence.py crea tablas y guarda lotes. tests/test_validation_and_persistence.py comprueba reglas, duplicados, catálogo y rollback. src/pipeline.py reúne candidatos y rechazos.

La función mostrada intenta construir el modelo. Si Pydantic lanza ValidationError, lo convierte en detalles y devuelve None junto con esos detalles. Así el código que recorre el lote puede continuar con la siguiente fila.

Fragmento real, líneas 281-288. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
281 def validate_atm_record_detailed(
282     raw: dict[str, Any],
283 ) -> tuple[ATMTransaction | None, list[dict[str, str]]]:
284     try:
285         model = ATMTransaction.model_validate(raw)
286         return model, []
287     except ValidationError as exc:
288         return None, _validation_error_details(exc)
```

Modelos y validadores por campo

Archivo: src/validation.py | Función o clase: DigitalTransaction | Inicio: línea 201

BaseModel declara los campos y tipos. @field_validator conecta una función con un campo. mode=before permite normalizar antes de la conversión de tipo. ATMTransaction representa campos del cajero; DigitalTransaction representa transfer_id, account_id, amount, currency y demás campos digitales.

_as_optional_decimal convierte importes y rechaza booleanos, texto corrupto y valores no finitos. Se permite monto ausente para que la imputación decida después. Consulta de saldo admite cero; las otras operaciones monetarias requieren monto positivo. Comisión admite cero, no negativos.

Los modelos de entrada permiten conversiones controladas, no strict=True global. AnaliticaFinal en src/transformacion.py sí usa ConfigDict(strict=True) para los valores calculados. No afirmes que todos los campos de todo el pipeline son estrictos de la misma manera.

Fragmento real, líneas 201-215. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
201 class DigitalTransaction(BaseModel):
202     model_config = ConfigDict(extra="ignore", str_strip_whitespace=True)
203 
204     transfer_id: str
205     timestamp: datetime
206     account_id: str
207     amount: Decimal | None = None
208     currency: CURRENCY
209     beneficiary_clabe: str
210     notification_email: EmailStr | None = None
211     concept: str = "SIN_CONCEPTO"
212     status: DIGITAL_ESTADO
213     platform: str
214 
215     @field_validator("transfer_id", "account_id", mode="before")
```

CLABE, correo, fechas y categorías

Archivo: src/validation.py | Función o clase: _validate_clabe | Inicio: línea 263

DigitalTransaction._validate_clabe exige exactamente 18 dígitos ASCII y conserva el dato como texto. Los ceros iniciales no se pierden. El formato no acredita que la cuenta bancaria exista ni comprueba el dígito de control de la CLABE.

notification_email es opcional: vacío o ausente queda None; si tiene contenido, EmailStr valida el formato. No se consulta que el buzón exista. _parse_digital_datetime admite ISO con zona o Unix en segundos; _parse_atm_datetime admite los formatos definidos y aplica UTC-06:00 cuando corresponde.

Moneda admite MXN y USD; categorías inválidas producen códigos estables. Fechas ambiguas, imposibles y números infinitos se rechazan. DESCONOCIDO y SIN_CONCEPTO son etiquetas controladas para algunos campos categóricos ausentes.

Fragmento real, líneas 263-267. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
263     def _validate_clabe(cls, value: Any) -> str:
264         text = "" if _missing(value) else str(value).strip()
265         if not re.fullmatch(r"[0-9]{18}", text, flags=re.ASCII):
266             raise ValueError("CLABE_INVALIDA")
267         return text
```

Errores y cuarentena con explicación

Archivo: src/validation.py | Función o clase: validate_digital_record_detailed | Inicio: línea 296

_validation_error_details traduce errores de Pydantic en codigo, campo y descripcion. normalizar, en src/integracion.py, envuelve estos detalles con el original y la fase VALIDACION. Los huérfanos se detectan después, en INTEGRACION; no todo rechazo viene de Pydantic.

insert_quarantine_record en src/persistence.py guarda original_record, motivo_rechazo_cuarentena, codigo_error, detalles_errores, fase y ejecución. Puede conservar varios motivos en una fila. estado_seguimiento permite distinguir ABIERTA de RESUELTA.

ATM-TX-00009 se rechaza por cuenta ausente. En cambio SPEI-2026-00003 puede estar limpia aunque su estado bancario sea RECHAZADA: limpia significa datos válidos, no operación aprobada. Nunca mezcles estos conceptos.

Fragmento real, líneas 296-303. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
296 def validate_digital_record_detailed(
297     raw: dict[str, Any],
298 ) -> tuple[DigitalTransaction | None, list[dict[str, str]]]:
299     try:
300         model = DigitalTransaction.model_validate(raw)
301         return model, []
302     except ValidationError as exc:
303         return None, _validation_error_details(exc)
```

Tablas y registros completos

Archivo: src/persistence.py | Función o clase: insert_clean_record | Inicio: línea 317

initialize_database crea el esquema y habilita claves foráneas. Las dos tablas obligatorias son transacciones_limpias y cuarentena_errores. También hay ejecuciones, evidencia_lotes, catálogo versionado y bitácora de omitidos para explicar lo ocurrido.

insert_clean_record escribe identificadores y payload. payload es texto JSON con el registro completo; _json_dumps serializa Decimal como cadenas para preservar exactitud. Para encontrar una moneda no busques necesariamente una columna SQL: usa json_extract(payload, ...).

La identidad de operación es fuente más ID. _source_fingerprint calcula la huella del contenido original normalizado. build_load_identity incorpora hashes de archivos y versiones. Mismo contenido evita insertar dos veces; mismo ID con contenido distinto va a cuarentena.

Fragmento real, líneas 317-328. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
317 def insert_clean_record(
318     connection: sqlite3.Connection,
319     fuente: str,
320     record: dict[str, Any],
321     execution_id: str | None = None,
322     catalog_version: str | None = None,
323 ) -> int:
324     origin = _source_record(record)
325     cursor = connection.execute(
326         """
327         INSERT INTO transacciones_limpias (
328             fuente, id_transaccion, id_cuenta, ejecucion_id, version_catalogo,
```

Commit y rollback: guardar todo el lote

Archivo: src/persistence.py | Función o clase: transactional_connection | Inicio: línea 272

El context manager abre una conexión y comienza BEGIN. yield entrega la conexión al bloque de guardado. Si termina bien, commit confirma. Si ocurre una excepción, rollback deshace las escrituras y raise conserva el error. finally cierra la conexión.

persist_batch registra primero la ejecución EN_PROCESO. Luego guarda aceptadas, cuarentena, omitidas, evidencia y cierre del lote en una transacción. Ante fallo, una conexión independiente registra FALLIDA. Esto evita que un error deje media carga guardada.

Una transacción SQL es un conjunto de escrituras atómicas; no es una transferencia bancaria. Los fallos de infraestructura pueden propagarse: lo probado es rollback y trazabilidad, no que el sistema nunca pueda detenerse por cualquier causa.

Fragmento real, líneas 272-282. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
272 def transactional_connection(db_path: str | Path) -> Iterator[sqlite3.Connection]:
273     connection = initialize_database(db_path)
274     try:
275         connection.execute("BEGIN")
276         yield connection
277         connection.commit()
278     except Exception:
279         connection.rollback()
280         raise
281     finally:
282         connection.close()
```

Pruebas necesarias para la rúbrica

Archivo: tests/test_validation_and_persistence.py | Función o clase: test_batch_rolls_back_and_records_failed_execution | Inicio: línea 294

Esta prueba crea una base temporal e instala un trigger que falla en ATM-SECOND. Intenta guardar dos filas. Comprueba que no quede ninguna limpia y que la ejecución sea FALLIDA. Espera sqlite3.IntegrityError: la excepción del caso es deliberada.

El mismo archivo verifica montos no finitos, CLABE, correo, fechas, faltantes imputables, duplicados, recargas y claves foráneas. tests/test_pipeline.py prueba originales, repetición y reproceso. tests/test_api.py conserva la prueba de XLSX/JSON corruptos antes ubicada en test_entrega.py.

Ejecutar: .venv/Scripts/python.exe -m pytest -q. La evidencia actual está en docs/evidencia/pruebas.txt. Eliminar estas pruebas impediría respaldar la exigencia de resiliencia del profesor; los casos sintéticos se crean en carpetas temporales, no en la base de entrega.

Fragmento real, líneas 294-308. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
294 def test_batch_rolls_back_and_records_failed_execution(tmp_path):
295     db_path = tmp_path / "rollback.sqlite"
296     initialize_database(db_path)
297     connection = sqlite3.connect(str(db_path))
298     connection.execute(
299         """
300         CREATE TRIGGER reject_second_transaction
301         BEFORE INSERT ON transacciones_limpias
302         WHEN NEW.id_transaccion = 'ATM-SECOND'
303         BEGIN
304             SELECT RAISE(ABORT, 'simulated persistence failure');
305         END;
306         """
307     )
308     connection.commit()
```

Cómo comprobar y defender la salida

Archivo: src/persistence.py | Función o clase: fetch_summary | Inicio: línea 629

La base oficial es entrega/pipeline_banco.sqlite. Las bases de la interfaz viven en outputs/ y pueden tener otros archivos. Para la entrega original: ATM 568 limpias, 32 cuarentena y 20 omitidas; DIGITAL 504 limpias y 96 cuarentena. Total: 1220 = 1072 + 128 + 20.

Ejecuta .venv/Scripts/python.exe docs/guias/consultar_ejemplos.py leo. Abre la base en solo lectura, muestra conteos, ejemplo de rechazo, PRAGMA integrity_check y foreign_key_check. Se espera ok y ninguna referencia incumplida.

En un visor SQLite: SELECT COUNT(*) FROM transacciones_limpias; y SELECT COUNT(*) FROM cuarentena_errores;. Explica dónde queda el original, cómo leer motivos y por qué una fila puede tener varios errores. Revisa el test de rollback y señala las líneas commit y rollback sin depender de memoria.

Fragmento real, líneas 629-640. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
629 def fetch_summary(db_path: str | Path) -> dict[str, int]:
630     connection = sqlite3.connect(str(db_path))
631     try:
632         clean_count = connection.execute(
633             "SELECT COUNT(*) FROM transacciones_limpias"
634         ).fetchone()[0]
635         quarantine_count = connection.execute(
636             "SELECT COUNT(*) FROM cuarentena_errores"
637         ).fetchone()[0]
638     finally:
639         connection.close()
640     return {"clean": clean_count, "quarantine": quarantine_count}
```

Repaso y archivos para entregar

ETL significa extraer, transformar y cargar. Puedes explicar tu módulo indicando su entrada, la decisión que toma, su salida y la evidencia que conserva. No basta con leer nombres de técnicas.

Los ejemplos de estas guías corresponden a datasets/catalogo_cuentas.csv, datasets/transacciones_atm.xlsx y datasets/transferencias_digitales.json. Con archivos nuevos cambian las estadísticas y resultados. La tasa académica USD/MXN es fija: 19.50.

Entrega al profesor: enlace al repositorio Git, entrega/pipeline_banco.sqlite y entrega/REPORTE_FINAL.pdf. Dentro del repositorio quedan reglas, diagrama, código y pruebas de resiliencia. Las guías de docs/guias son material de estudio del equipo.

El video y la presentación indicados posteriormente por el profesor se preparan aparte; esta guía no equivale a haber grabado o expuesto. docs/GUION_VIDEO.md propone el recorrido. La interfaz es un apoyo visual para demostrar el pipeline.

Para localizar el código, abre el archivo .py en el editor y busca el nombre de función indicado. Las líneas corresponden a esta versión y pueden moverse con futuros cambios. Los fragmentos se extraen del código al regenerar las guías; no son pseudocódigo.

Referencias: guía del profesor, puntos 4 y 6; REGLAS_DE_NEGOCIO.md; docs/CONTRATO_DATOS.md; código y pruebas citados; docs/evidencia/resultados.json; base SQLite de entrega. Regla vigente 1.5, proceso 2.1.